COPY OF SIMPLIFIER POC - INSTALLATION REQUIREMENTS

created by:	approved by:
unit:	issue date:
revision:	document:
confidentiality:	

STORM REPLY
Bartholomäusweg 26 · D-33334 Gütersloh · Deutschland
storm@reply.de

www.reply.de

CONTENTS

1 Copy of Simplifier PoC - Installation Requirements

2 Introduction

This document separates:

1. **Customer preparation** that can be completed before the AWS DevOps Agent is deployed.
2. **Deployment and integration by REPLY** after the customer preparation is complete.

3 Setup Guide

3.1 Azure DevOps

To integrate the agent with the customer's **Azure DevOps organization**, the solution requires:

- an **Azure DevOps MCP server**, which acts as the communication layer between the agent and Azure DevOps;
- a dedicated **non-human identity**, such as a **Microsoft Entra application**, used by the solution for authentication;
- the necessary **Azure DevOps organization access and permissions**, granted and managed by the customer.

Using a non-human identity avoids dependency on individual user accounts, interactive login sessions or employee lifecycle events, such as role changes or account deactivation.

For the Azure DevOps MCP server, two deployment options are available: a remote server hosted by Microsoft and a local MCP server deployed as part of the solution runtime. The remote Azure DevOps MCP server is mainly designed for interactive Microsoft Entra OAuth usage with end-user tools such as Visual Studio Code. For this reason, it is less suitable for a dedicated service-to-service integration. In this solution, we use the local MCP server, which is already embedded in our architecture and supports authentication mechanisms suitable for non-human access. This option is also preferred because it provides a more stable and predictable release lifecycle, reducing the risk of breaking changes affecting the integration.

To enable the integration, the customer needs to prepare the following items:

3.1.1 Required Azure DevOps Information

- Azure DevOps organization name;
- if applicable, the default target project to be made available to the agent.

Access is expected to apply to the resources associated with the selected project(s), according to the permissions assigned by the customer within Azure DevOps.

3.1.2 Microsoft Entra Application Configuration

The customer should register a dedicated Microsoft Entra application to be used as the non-human identity for the Azure DevOps integration.

The application should be configured as follows:

1. Create a new App registration in Microsoft Entra ID
2. Use a clear and dedicated name, for example: StormAI DevOps Agent
3. Configure the application as single-tenant

4. No Redirect URI is required since the application is used only for non-interactive service-to-service authentication
5. Create a Client Secret for the application

Important: the client secret value is visible only once, immediately after creation. It must be copied and stored securely at that time

6. Record the following values:

- – Tenant ID;
  – Client ID / Application ID;
  – Client Secret

7. No broad Microsoft Graph permissions are required unless explicitly needed for a separate use case.

3.1.3 Add the Microsoft Entra Application to the Azure DevOps Organization

The Microsoft Entra application registration alone is not sufficient: the identity must also be recognized as a user inside Azure DevOps and assigned the appropriate access.

The customer needs to:

1. Open the target Azure DevOps organization
2. Go to Organization settings
3. Open Users
4. Select Add users
5. Search for the service principal using the registered application name, for example StormAI DevOps Agent
6. Assign the required access level
7. Add it to the target project(s)
8. Configure the required project permissions according to the agreed scope

Once added, the service principal can be managed like a technical user within Azure DevOps, including project membership, repository permissions, board access and other organization-level controls.

3.1.4 Azure DevOps Integration Configuration

The required Azure DevOps integration values can be provided through the **OpenTofu tfvars file** used for the deployment.

Example:

```
azure_devops_config = {
  org_name  = "<AZURE_DEVOPS_ORG_NAME>"
  project   = "<AZURE_DEVOPS_PROJECT_NAME>"
  tenant_id = "<AZURE_ENTRA_TENANT_ID>"
}
```

After the Microsoft Entra application has been created and the solution deployed on AWS, update the Secret Manager entry with the actual **Client ID** and **Client Secret**.

The secret name can be retrieved from the OpenTofu output:

```
tofu output -raw azure_devops_client_credentials_secret_name
```

Then update the secret value:

```
aws secretsmanager put-secret-value \
  --region <AWS_REGION> \
  --secret-id "<SECRET_NAME_FROM_OUTPUT>" \
  --secret-string '{"client_id":"<AZURE_CLIENT_ID>","client_secret":"<AZURE_CLIENT_SECRET>"}'
```

3.2 AWS

3.2.1 Responsibilities at a glance

Phase	Owner	Result
Provide central parameters	REPLY	Monitoring account ID, Region, and agreed role name
Create target-account IAM role	Customer	One assumable role in each AWS account to be monitored
Deploy AWS DevOps Agent	REPLY	Agent Space and primary-account association
Connect target accounts	REPLY	Each customer role is associated with the Agent Space
Validate and optionally harden trust	REPLY and customer	Active connection; optional restriction to the final Agent Space ID

The customer does **not** need to deploy an AWS DevOps Agent or know the final Agent Space ID before completing the preparation below.

3.3 Customer preparation before deployment

3.3.1 Scope

Create one IAM role in every customer AWS account that should be monitored. AWS DevOps Agent will later assume this role directly through the AWS service principal `aidevops.amazonaws.com`.

No long-lived AWS access keys, secret keys, or session tokens must be shared with StormAI or REPLY.

The customer supplies:

- `<MONITORING_ACCOUNT_ID>` – The 12-digit AWS account ID in which StormAI/REPLY will deploy the Agent Space.
- `<REGION>` – The AWS Region in which the Agent Space will be deployed.
- The role name, normally `DevOpsAgentCrossAccountRole`.
- `<EXTERNAL_ACCOUNT_ID>` – The 12-digit ID of each customer account to be monitored.
- The ARN of the role created in each account.

3.3.2 Customer prerequisites

The customer needs:

- Administrative access, or sufficient IAM permissions to create and configure a role in each target account.
- Confirmation that AWS Organizations SCPs and permission boundaries allow the required IAM operations.
- A normal change record or approval for attaching the AWS-managed `AIDevOpsAgentAccessPolicy`.

`AIDevOpsAgentAccessPolicy` is maintained and versioned by AWS and can change over time. The customer should review the current AWS-managed policy as part of the approval.

3.3.3 Verify the target account

Run the following command with credentials for the customer account in which the role will be created:

```
aws sts get-caller-identity
```

Confirm that the returned account ID equals `<EXTERNAL_ACCOUNT_ID>`.

3.3.4 Create the pre-deployment trust policy

Because the Agent Space does not yet exist, its final ID is not available. The trust policy therefore permits Agent Spaces only:

- From the designated monitoring account, enforced by `aws:SourceAccount`.
- In the agreed Region.
- Under the `agentspace/*` resource type.

This follows the same pre-deployment trust pattern AWS uses when an Agent Space ID is not yet available.

```
cat > devops-cross-account-trust-policy.json << 'EOF'
```

3.3.5 Attach the AWS-managed access policy

```
{
  "Version": "2012-10-17",
  "Statement": [
      {
      "Effect": "Allow",
      "Principal": {
        "Service": "aidevops.amazonaws.com"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": {
          "aws:SourceAccount": "<MONITORING_ACCOUNT_ID>"
        },
        "ArnLike": {
          "aws:SourceArn": "arn:aws:aidevops:<REGION>:<MONITORING_ACCOUNT_ID>:agentspace/*"
        }
      }
    }
  ]
```

```
}
EOF
```

```
aws iam create-role \
  --role-name DevOpsAgentCrossAccountRole \
  --assume-role-policy-document file://devops-cross-account-trust-policy.json
```

```
aws iam attach-role-policy \
  --role-name DevOpsAgentCrossAccountRole \
  --policy-arn arn:aws:iam::aws:policy/AIDevOpsAgentAccessPolicy
```

The effective permissions are the intersection of the role permissions and the session guardrail applied by AWS DevOps Agent.

3.3.6 Allow creation of the Resource Explorer service-linked role

This permission lets AWS DevOps Agent create the Resource Explorer service-linked role in the customer account if it does not already exist.

```
cat > devops-cross-account-additional-policy.json << 'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCreateServiceLinkedRoles",
      "Effect": "Allow",
      "Action": "iam:CreateServiceLinkedRole",
      "Resource": "arn:aws:iam::<EXTERNAL_ACCOUNT_ID>:role/aws-service-role/resource-explorer-2.amazona
      ↪  ws.com/AWSServiceRoleForResourceExplorer"
    }
  ]
}
EOF

aws iam put-role-policy \
  --role-name DevOpsAgentCrossAccountRole \
  --policy-name AllowCreateServiceLinkedRoles \
  --policy-document file://devops-cross-account-additional-policy.json
```

3.3.7 Validate and hand over the result

```
aws iam get-role \
  --role-name DevOpsAgentCrossAccountRole \
  --query 'Role.Arn' \
  --output text

aws iam list-attached-role-policies \
  --role-name DevOpsAgentCrossAccountRole

aws iam get-role-policy \
  --role-name DevOpsAgentCrossAccountRole \
  --policy-name AllowCreateServiceLinkedRoles
```

Provide REPLY with only:

- External account ID

- Role ARN

Repeat steps 1–5 for every customer account to be monitored.

3.3.8 Terraform/OpenTofu example for the customer role

Run this configuration with an AWS provider authenticated to the customer target account. The target account ID is derived from the active provider identity.

```
terraform {
  required_version = ">= 1.6.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

variable "monitoring_account_id" {
  type        = string
  description = "The AWS account ID in which StormAI/REPLY will deploy the Agent Space"

  validation {
    condition     = can(regex("^\\d{12}$", var.monitoring_account_id))
    error_message = "The monitoring_account_id must be a valid 12-digit AWS account ID."
  }
}

variable "agent_space_region" {
  type        = string
  description = "The AWS Region in which StormAI/REPLY will deploy the Agent Space"
}

data "aws_caller_identity" "target" {}

resource "aws_iam_role" "devops_agent_cross_account" {
  name = "DevOpsAgentCrossAccountRole"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Principal = {
          Service = "aidevops.amazonaws.com"
        }
        Action = "sts:AssumeRole"
        Condition = {
          StringEquals = {
            "aws:SourceAccount" = var.monitoring_account_id
          }
          ArnLike = {
            "aws:SourceArn" =
            ↪  "arn:aws:aidevops:${var.agent_space_region}:${var.monitoring_account_id}:agentspace/*"
          }
        }
      }
```

```
    ]
  })

  tags = {
    Purpose   = "AWS DevOps Agent Cross-Account Access"
    ManagedBy = "Terraform"
  }
}

resource "aws_iam_role_policy_attachment" "devops_agent_access" {
  role       = aws_iam_role.devops_agent_cross_account.name
  policy_arn = "arn:aws:iam::aws:policy/AIDevOpsAgentAccessPolicy"
}

resource "aws_iam_role_policy" "allow_resource_explorer_service_linked_role" {
  name = "AllowCreateServiceLinkedRoles"
  role = aws_iam_role.devops_agent_cross_account.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Sid      = "AllowCreateServiceLinkedRoles"
        Effect   = "Allow"
        Action   = "iam:CreateServiceLinkedRole"
        Resource = "arn:aws:iam::${data.aws_caller_identity.target.account_id}:role/aws-service-role/re
        ↪  source-explorer-2.amazonaws.com/AWSServiceRoleForResourceExplorer"
      }
    ]
  })
}

output "external_account_id" {
  value = data.aws_caller_identity.target.account_id
}

output "role_arn" {
  value = aws_iam_role.devops_agent_cross_account.arn
}
```
